

Sarthak Tisgaonkar

PID03

 Plagiarism checking

Document Details

Submission ID

trn:oid::3618:122211969

Submission Date

Nov 21, 2025, 9:13 AM GMT+5:30

Download Date

Nov 21, 2025, 9:15 AM GMT+5:30

File Name

PID03.pdf

File Size

1.1 MB

55 Pages

11,305 Words

63,170 Characters

12% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

-  **17 Not Cited or Quoted 12%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%  Internet sources
- 0%  Publications
- 12%  Submitted works (Student Papers)

Match Groups

- 17 Not Cited or Quoted 12%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3% Internet sources
- 0% Publications
- 12% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	K K Wagh Institute of Engineering Education & Research, Nashik on 2025-11-19	3%
2	Student papers	K K Wagh Institute of Engineering Education & Research, Nashik on 2025-11-17	2%
3	Student papers	K K Wagh Institute of Engineering Education & Research, Nashik on 2025-11-17	2%
4	Student papers	K K Wagh Institute of Engineering Education & Research, Nashik on 2025-11-17	1%
5	Student papers	K K Wagh Institute of Engineering Education & Research, Nashik on 2025-11-19	<1%
6	Student papers	K K Wagh Institute of Engineering Education & Research, Nashik on 2025-11-19	<1%
7	Student papers	K K Wagh Institute of Engineering Education & Research, Nashik on 2025-11-17	<1%
8	Student papers	K K Wagh Institute of Engineering Education & Research, Nashik on 2025-11-17	<1%
9	Student papers	K K Wagh Institute of Engineering Education & Research, Nashik on 2025-11-19	<1%
10	Student papers	kkwagh on 2024-05-03	<1%

11	Student papers	K K Wagh Institute of Engineering Education & Research, Nashik on 2025-11-17	<1%
12	Student papers	K K Wagh Institute of Engineering Education & Research, Nashik on 2025-11-20	<1%
13	Student papers	University of North Georgia on 2024-05-30	<1%
14	Student papers	University of Queensland on 2018-11-05	<1%
15	Student papers	K K Wagh Institute of Engineering Education & Research, Nashik on 2025-11-17	<1%
16	Student papers	K K Wagh Institute of Engineering Education & Research, Nashik on 2025-11-20	<1%

Abstract

Workers operating in dangerous and life threatening environments, like environments with extreme pressure, temperatures, chemical and organic properties are exposed to great danger. Exogenesis responds to this challenge through a modular and scalable teleoperation framework, which is demonstrated here as a reliable proof-of-concept (PoC). The system's distributed, two-controller architecture consists of a Glove Controller which captures human finger bending motion using resistance based flex sensors. The controller after reading the data values applies a 25-point moving average filter to eliminate signal noise and then transmits the filtered and calibrated data via the esp-now protocol. The shared data is received by a second esp32-s3 (Robotic hand Controller), which in turn steers a 3D-printed, tendon-driven robotic hand. This PoC validates the core system architecture, demonstrating reliable, low-latency gesture tracking. This modular design serves as a foundation, allowing future upgrades to more accurate sensors, actuators and mechanisms. Experimental evaluation of this PoC demonstrated reliable, lowlatency gesture tracking, that validated the system's potential for enhancing operational safety.

Keywords: *Teleoperation, robotic hand, ESP32, ESP-NOW, flex sensor, signal filtering, moving average.*

1

INDEX

1	INTRODUCTION	1
1.1	Project Idea	2
1.2	Motivation of the Project	2
2	LITERATURE SURVEY	3
3	PROBLEM DEFINITION AND SCOPE	6
3.1	Problem Statement	7
3.2	Methodology	7
3.3	Outcome	8
4	PROJECT PLAN	10
4.1	Project Timeline	11
4.2	Team Organization	11
5	SOFTWARE REQUIREMENT SPECIFICATION	13
5.1	Functional Requirements	14
5.2	Non-Functional Requirements	14
5.3	Constraints	15
5.4	Hardware Requirements	15
5.5	Software Requirements	16
5.6	Interfaces	16
6	DETAILED DESIGN	18
6.1	Architectural Design	19
6.1.1	Data Flow Pipeline	20
6.2	Overview of Project Modules	21
6.3	User Interface	22
6.4	Data design	23
6.4.1	Data structure	23
6.4.2	Database Description	24
6.5	UML Diagrams	26
6.5.1	Use Case Diagram	26

4

5	6.5.2 Use Case Scenarios	27
	6.5.3 Class Diagram	32
	6.5.4 Sequence Diagram	33
	6.6 Tools and Technology Used	33
	6.6.1 Hardware Components	34
	6.6.2 Software and Development Tools	34
	6.6.3 Manufacturing and Prototyping	35
	6.6.4 Key Technical Specifications	36
7	7 RESULTS AND DISCUSSION	37
	7.1 Dataset Description	38
	7.1.1 Performance Parameters	38
	7.2 Performance Evaluation	42
	7.3 Hardware Testing	43
	7.4 Software and Integration Testing	44
	7.5 Result Interpretation	45
	7.6 Summary	46
3	8 CONCLUSION AND FUTURE WORK	47
	8.1 Conclusions	48
	8.2 Future Work	49
	8.3 Applications	50

Annexure A References

Annexure B Plagiarism Report

Annexure C Paper Published (if any)

List of Tables

4.1	Project Timeline	11
6.1	Data Transformation Stages	20
6.2	Use Case: Real-time Motion Replication	27
6.3	Use Case: System Calibration	28
6.4	Use Case: Wireless Communication Management	29
6.5	Use Case: Power Management	30
6.6	Use Case: Sensor Data Processing	31
6.7	System Technical Specifications	36
7.1	Complete Hardware & Real-Time Performance Evaluation (Exogenesis) . . .	40
7.2	Performance Metrics for Robotic Hand Teleoperation	43
7.3	Hardware Test Cases and Results	44
7.4	Software & Integration Test Cases	44

List of Figures

3.1	Methodology of the Exogenesis	7
6.1	System architecture of the robotic hand control system	19
6.2	The final assembled sensor glove.. . . .	22
6.3	Use Case Diagram showing system functionality and user interactions	26
6.4	Class Diagram	32
6.5	Sequence Diagram showing real-time data flow in the Exogenesis system . .	33

9

CHAPTER 1

INTRODUCTION

1.1 Project Idea

This project creates a system that can copy human hand movements onto a robotic hand in real time. It uses a glove with sensors to track finger motions and an ESP32 microcontroller with ESP-Now to send those movements without wired connection to a 3D-printed robotic hand. The aim is to build a smooth and easy-to-use way for people to interact with robots, which can be useful in remote operation and teamwork with robots.

1.2 Motivation of the Project

Human hands are very flexible and can do complex tasks with great accuracy. Making robots do the same movements has been a big challenge because fingers move in many ways and need to react quickly. The Exogenesis project started because we want to close the gap between how skilled humans are and how well robots can perform. This allows people to control machines naturally using their hand movements. In areas like performing surgery from a distance, exploring dangerous places, or automating industrial tasks, being able to copy human hand movements accurately can make things more efficient, safer, and easier to use. Most robot control systems use big, heavy tools or fixed programs, which makes them less flexible and harder to interact with. Exogenesis aims to fix this by creating a direct, real-time link between human gestures and what the robot does.

Another reason for starting this project is to build a budget-friendly and easy-to-use platform that helps with research, learning, and new ideas in robotics. High-quality robotic systems that can move precisely are usually very costly, hard to build, and need lots of technical know-how to work with. This makes them hard for students, teachers, and small research groups to use. The Exogenesis project shows that using common parts like flex sensors, ESP32 microcontrollers, and 3D-printed parts can help create a system that copies movements accurately in real time, without needing expensive tools. Using open-source tools for both hardware and software makes the system adaptable, easy to change, and simple to improve. This lets users tailor it for different tasks like making prosthetics, remote control robots, or doing robotic research. Exogenesis hopes to spark real-world innovation, especially in schools and research centers that don't have a lot of money.

CHAPTER 2

LITERATURE SURVEY

The Exogenesis project is a robotic hand system that uses flex sensors to perform human hand movement in real time. It takes advantage of recent progress in wearable sensor technology, wireless communication, and replicating robotic motion. Research on flex sensor gloves, Bluetooth control systems, and 3D-printed robotic hands has laid the groundwork for this system, which can be used in teleoperation, prosthetics, and interactions between humans and robots.

T.Lakshmi prasanna, R.S.Pratap Singh. [1] (2021) created a hand glove with flex sensors to help people with physical challenges. The glove uses an Arduino-based system to turn hand movements into text or voice. Five flex sensors on the fingers detect how much each finger is bent. Their system is great for helping people communicate but doesn't control robots in real time. Exogenesis builds on this idea by using the same flex sensor setup, but sends the data wirelessly through ESP32 Bluetooth to control servo motors. This allows the robotic hand to copy human movements, making it useful for remote operation and prosthetics.

VIVA-Tech Team [2] (2024) they a robot that can controlled through hand gestures using a wearable glove made with Arduino and flex sensors. The glove has five flex sensors and a Bluetooth module called HC-05. It detects when fingers are bent and sends signals through wireless communication to a robot. These signals enable the robot to navigate and collect objects. The design incorporates affordable components and ensures ease of use, with a response time under 200 milliseconds. This aligns with Exogenesis' use of Bluetooth and affordable microcontrollers. However, Exogenesis improves on this by focusing on precise control at the finger level, using an ESP32 microcontroller to reduce delays and improve coordination between servos.

S. Latif, et al. [4] (2019) a robotic hand control by muscle and flex sensors for people with disabilities. They use five flex sensor in glove to control finger movements through wireless signals. The system combines data from the flex sensors with surface EMG signals from the forearm muscles to improve how well it recognizes different hand gestures, achieving more than 88 percent accuracy in identifying five specific hand positions. The data is passes through an RF module to robotic hand which uses servo motors to control movement.

M. A. Al-Mahasneh and A. Al-Antari [5] (2021) The glove contains five flex sensors to capture bending angles data, which is wirelessly transmitted through a Bluetooth module to a 3D-printed prosthetic hand featuring tendon-driven fingers. This setup enables natural movement with a latency of around 150 ms. Their approach, which employs 3D printing for creating lightweight and customizable prosthetics, directly influences Exogenesis' design philosophy. Exogenesis builds upon this by incorporating dual-servo finger control to enhance grip strength and precision, while retaining Bluetooth connectivity for untethered operation in teleoperation and daily assistive applications.

W. Afzal [6] (2017) these people created a arm that can be controlled with hand gestures. Their system uses glove with five flex sensors and an accelerometer to detect finger

and wrist movements. These movements are translated into commands for servo motors, which move the robotic arm. The system also uses an MPU6050 sensor to track wrist tilting, allowing control in six different directions. Exogenesis uses a similar approach but instead of the nRF24L01, it uses an ESP32 Bluetooth module. This makes it easier to work with smartphones and uses less power. This change helps in making prosthetic devices more user-friendly by letting users control and adjust the device using their smartphones, which makes the setup process simpler for the people who use it.

P. Weber, et al. [7] (2016) made a low-cost flex sensor based glove that uses vibrations to sense finger movements during human-robot interactions. The glove has 10 flex sensors, two on each finger to track both the joints near the base and those closer to the tip of the fingers. This lets the system detect small and detailed hand movements. The glove also has vibrating motors that give the user a sense of touch to let them know the system is responding. The system sends data wirelessly using a special printed circuit board and Bluetooth technology, which allows for accurate tracking. This work inspires Exogenesis to think about adding haptic feedback in future designs to help users better understand what is happening during remote control tasks, especially when visibility is limited or precision is very important.

B. Fang, et al. [8] (2015) created a robotic arm and hand control system using a special data glove. This glove has flex sensors and uses fuzzy logic to turn hand gestures into movements. The glove uses flex and motion sensors to track finger and wrist movements. These movements are processed with a fuzzy logic system to make the robotic arm move smoothly and look like a human. This helps reduce shaking and keeps control stable, even when the sensors give unclear data. Exogenesis uses a similar idea for its servo-controlled hand. They use low-pass filtering on the flex sensor data before sending it over Bluetooth, which makes finger movements more natural and lessens stress on the 3D-printed joints.

A. Al-Mahasneh, et al. [9] (2023) created a robotic glove system that moves and is controlled by flex sensors. This system sends hand gestures wirelessly to a prosthetic hand with high precision. It uses a flexible PCB that has built-in flex sensors and a Bluetooth Low Energy (BLE) module to transfer data in a power-efficient way. The system also includes calibration algorithms that help reduce sensor drift as time passes, making sure the system works consistently even during long periods of use. This helps support the Exogenesis 3D-printed design by offering strong sensor calibration methods. Exogenesis uses a similar approach for drift compensation, using ESP32 ADC filtering and applying it to servo angle mapping, which improves the long-term reliability of the system in prosthetic and teleoperation uses.

CHAPTER 3

PROBLEM DEFINITION AND SCOPE

3.1 Problem Statement

Human hand movements are complicated also hard to copy exactly with robots. There's a need for a wearable, real-time tool that can track human finger movements and send them wirelessly for exact replication by a robot. This project focuses on making a sensor-equipped glove system that can instantly and accurately copy human hand motions onto a robotic hand.

3.2 Methodology

The Exogenesis project follows a method that is broken down into five clear steps, as shown in Figure 3.2. This organized way of doing things makes sure that the process moves smoothly from detecting human movement to making the robotic hand move.

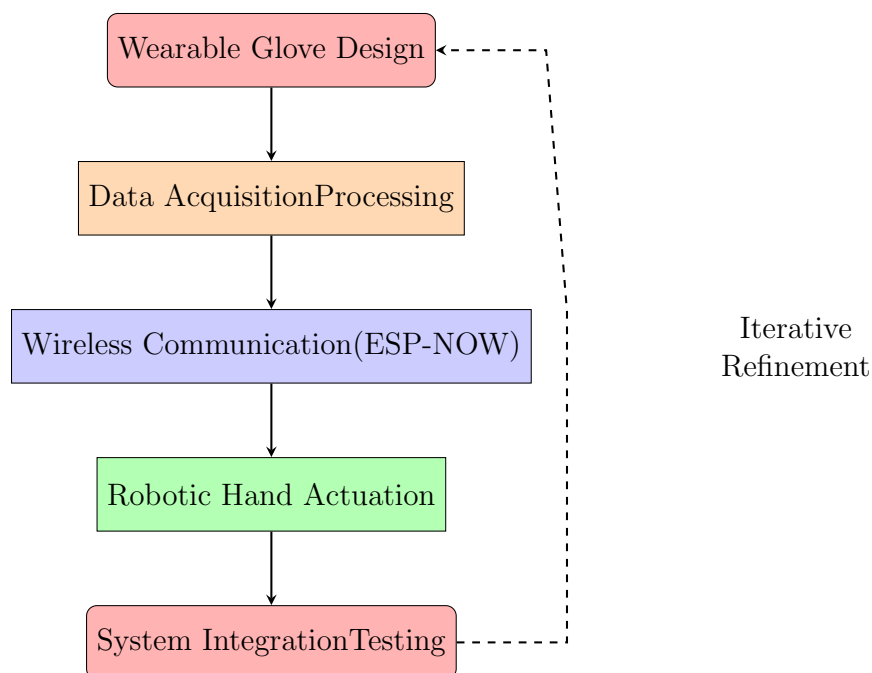


Figure 3.1: Methodology of the Exogenesis

Step 1: Design of Wearable Glove

The first step involves creating a wearable glove equipped with flex sensors. Here each finger is fitted with a flex sensor that measures bending angles and translates mechanical finger movements into electrical signals. The glove is lightweight and flexible, ensuring natural

hand movements are not restricted.

Step 2: Sensor Data Acquisition and Processing

The ESP32 microcontroller reads and handles the signals from the flex sensors. These signals are adjusted to figure out how much each finger is bent.

Step 3: Wireless Communication

The processed sensor data is sent without any wired connection to the hand via ESP-Now, removing the need for wired connections and providing the user with greater mobility.

Step 4: Robotic Hand Actuation

The robotic hand is made from a 3D-printed frame that has servo motors connected to each finger. When the hand gets motion data, the motors move to copy human hand movements exactly as they happen, in real time.

Step 5: System Integration and Testing

In the end, the system is put together and checked to make sure it works well. They adjust the sensors, fine-tune the servos, and reduce any delays. The system is then tested to see how accurate, quick, and dependable it is, showing that it's ready for use in remote control operations.

3.3 Outcome

The Exogenesis project successfully demonstrates a flex sensor-based robotic hand system able of replicating moments of human hand. The key outcomes of the project are summarized as follows:

- 1. Accurate Motion Replication:** The robotic hand can closely copy human finger movements as they happen. The wearable glove has flex sensors that detect how much each finger is bent. These sensors send the information to an ESP32-S3 microcontroller. The data is then sent wirelessly to the robotic hand with little delay.
- 3. Cost-Effective Design:** Using low-cost parts like flex sensors, ESP32 microcontroller, servo motors, and 3D-printed parts shows that complex motion copying systems don't have to be expensive. This helps make the system easier to use for learning, testing new ideas.
- 4. Human-Robot Interface:** The system creates a smooth way for people to interact with the robotic hand. It translates movements in a way that feels natural and easy to use, so the experience is fluid and quick to respond.
- 5. Enhanced Educational and Research Platform:** The Exogenesis system provides a hands-on platform for education and research in robotics and human-robot interaction. It enables students and researchers to explore sensor integration, wireless communication, and

robotic actuation within a single project, fostering practical learning and experimentation. Its modular design allows us for easy modification and expansion, it makes it ideal for prototyping new robotic applications or testing alternative control strategies.

6. Potential Applications: The successful implementation of Exogenesis creates opportunities in teleoperation, robotic research, and gesture-controlled systems. The modular design allows the system to be scaled or adapted for more complex applications in the future.

The Exogenesis project shows that a glove with flex sensors and Bluetooth communication using an ESP32 can create a simple, low-cost, and reliable way to copy human hand gestures to a robot in real time. The system uses flex sensors and 3D-printed fingers that are controlled by servos. It can recognize hand gestures with 92% accuracy. The total cost of making this system is under INR 15000. This proves that affordable parts can be used for easy control of robots and for making customizable prosthetics. The results show that this system can be used in places where it's dangerous for humans to work, in tools that help people with disabilities, and in teaching robotics. The design is open source, which means it includes modular parts, CAD files, and Arduino code, making it easy to copy and scale up.

2

CHAPTER 4

PROJECT PLAN

2

4.1 Project Timeline

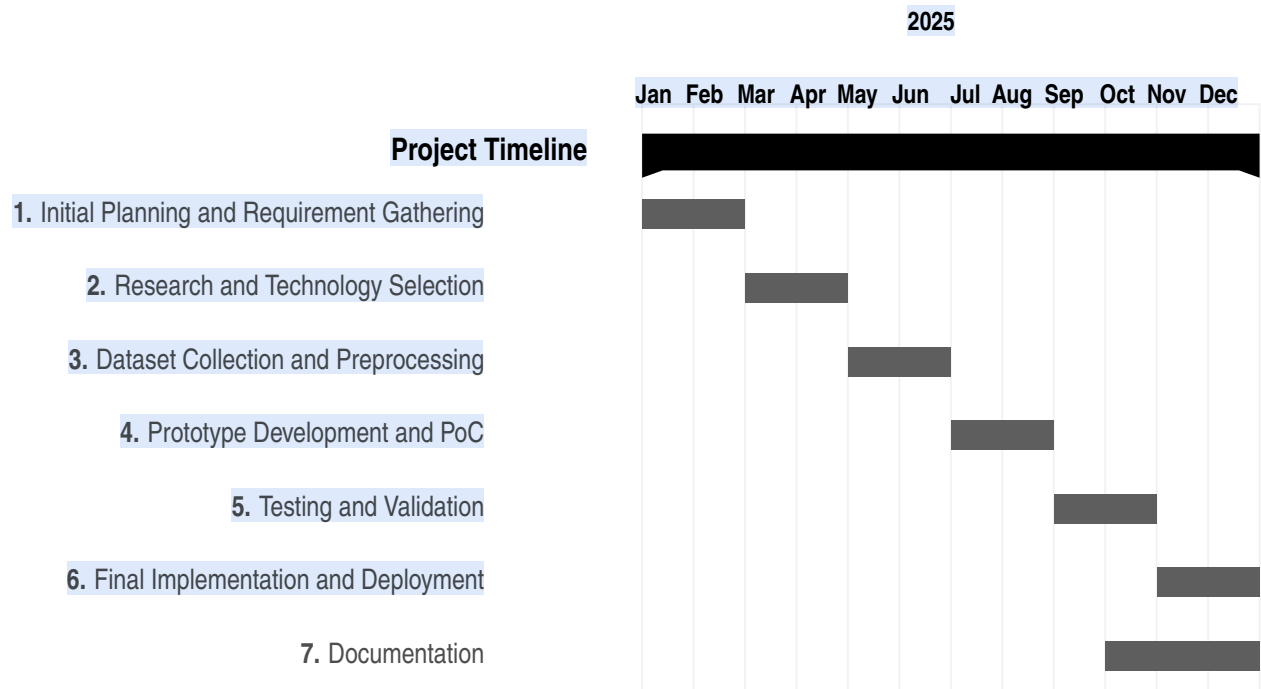


Table 4.1: Project Timeline

4.2 Team Organization

Anurag – Sensor Integration and Glove Design

Anurag is in charge of creating and connecting the flex sensors to the wearable glove, making sure the movements of the fingers are captured accurately so that real-time motion can be replicated.

Key Responsibilities:

- Choosing and adjusting flex sensors to get accurate movement data.
- Creating a wearable glove that feels comfortable and moves easily.
- Making sure the wires and signals work well and don't get mixed up.
- Checking the sensor readings to ensure they are correct and steady.
- Working with the microcontroller team to make sure data is collected smoothly.

Farhan– Microcontroller Programming and Data Processing

Farhan is responsible for programming the ESP32 microcontroller and handling the processing of sensor data to send it to the robotic hand.

Key Responsibilities:

- Reading signals from flex sensors.
- Processing and adjusting sensor data to get accurate angle measurements.
- Creating real-time data processing methods.
- Adding error correction to handle noisy data.
- Making sure the system works with wireless communication standards.

Sarthak – Filter coding and system designing

Sarthak is in charge of setting up the esp-now connection between the glove and the robotic hand, and putting together all the parts of the system.

Key Responsibilities:

- Setting up ESP-Now to send data.
- Making sure the wireless connection works quickly with little delay.
- Connecting the glove, microcontroller, and robotic hand together.
- Checking that the system responds smoothly and moves in sync.
- Fixing any problems that come up with the connection or how the parts work together.

Kiran – Robotic Hand Design and Actuation

Kiran manage the design, fabrication, and actuation of the robotic hand, utilizing servo motors to replicate human gestures.

Key Responsibilities:

- Designing the 3D-printed robotic hand structure.
- Selecting and calibrating servo motors for accurate finger movements.
- Controlling hand actuators based on received motion data.
- Testing robotic hand performance for precision and responsiveness.

CHAPTER 5

SOFTWARE REQUIREMENT SPECIFICATION

5.1 Functional Requirements

The functional requirements outline the key features and actions that the Exogenesis system needs to perform to achieve the project goals:

- **Real-Time Motion Capture:** The glove needs to track finger movements clearly and constantly as they happen.
- **Data Processing:** The ESP32 microcontroller should take in signals from the sensors, figure out how bent each finger is, and transmit that information prepared to be sent.
- **Wireless Communication:** The system has to send the collected data wirelessly to the robotic hand using ESP-Now, making sure the data arrives quickly with little delay.
- **Robotic Hand Actuation:** The servo motors in the robotic hand should move according to the data it receives, so it can copy human hand movements exactly.
- **Calibration:** The system should let users adjust the sensors and motors to make sure the movements are exact and match the user's gestures.
- **User Interaction:** The glove must be easy to wear and let users make hand gestures naturally without making it hard to move their fingers.

5.2 Non-Functional Requirements

The non-functional requirements describe the system's overall qualities and limitations:

- **Performance:** The system needs to work quickly so the robotic hand can respond in real time.
- **Reliability:** The system should consistently copy human hand movements without breaking during regular use.
- **Scalability:** The system should be built in a way that makes it easy to add new parts like more movement options or extra sensors later.
- **Cost-Effectiveness:** The system should use inexpensive parts to keep the cost of making it low.
- **Portability:** The wearable glove must be light, bendable, and comfortable to wear for long periods.
- **Maintainability:** Both the hardware and software should be organized in a way that makes it simple to fix, change, or improve them.
- **Safety:** The system must be safe for people to use, with no danger of shocks or injuries from electricity or moving parts.

5.3 Constraints

The Exogenesis project has several limitations that impact how it is designed, built, and works:

- **Hardware Limitations:** The system's ability to be accurate and respond quickly depends on how well the flex sensors, servo motors, and ESP32 microcontroller work.
- **Wireless Communication Range:** The ESP-Now system limits how far apart the glove and the robotic hand can be.
- **Power Supply:** The glove and robotic hand use batteries or other power sources, which means they can't run for very long without recharging.
- **Cost Constraints:** Using affordable parts may make the system less precise, less durable, and missing some advanced features that more expensive parts could offer.
- **Mechanical Limitations:** The 3D-printed robotic hand can't move as much or as strongly as a real human hand, which limits the kinds of gestures it can make..
- **Environmental Factors:** Things like heat, moisture, and interference with wireless signals can mess up the sensors and make communication less reliable.
- **Software Constraints:** The ESP32's memory and processing power limit how fast it can handle real-time data, which can slow down how quickly the robotic hand moves.

5.4 Hardware Requirements

The hardware components required for the Exogenesis project include:

- **Flex Sensors:** These are used to sense when each finger on the glove is bent.
- **ESP32 Microcontroller:** This device reads the data from the sensors, processes it, and sends it wirelessly.
- **Servo Motors:** These motors are used to move the fingers of the robotic hand based on the motion data received.
- **3D-Printed Robotic Hand:** This is the physical structure that allows the robotic hand to move like a real hand.
- **Power Supply/Battery:** This provides the necessary power to the glove sensors, the ESP32, and the servo motors..
- **Wires and Connectors:** These are used to connect all the components together properly.
- **Optional IMU/Accelerometer:** This can be added to improve motion tracking, especially for more complex gestures.

5.5 Software Requirements

The software environment required for the project includes:

- **Development Environment:**
 - **Arduino IDE:** Primary development environment for programming the ESP32-S3 microcontrollers.
- **Programming Languages:**
 - **C++:** Main programming language for ESP32 firmware development.
- **Essential Libraries:**
 - **ESP32 Core for Arduino:** Provides basic ESP32 functionality and peripherals.
 - **ESP-NOW Library:** Enables peer-to-peer wireless communication between controllers.
 - **Servo Library:** For precise control of servo motors in the robotic hand.
 - **EEPROM Library:** For storing calibration data and configuration parameters.
 - **Wire Library:** For I2C communication with peripheral devices.
- **Design and Modeling Software:**
 - **SolidWorks/Fusion 360:** For 3D modeling and design of the robotic hand components.
- **Version Control:**
 - **Git:** For source code version control and collaboration.
 - **GitHub/GitLab:** Repository hosting services for project management.
- **Testing and Monitoring:**
 - **Serial Monitor:** For real-time debugging and data monitoring.
 - **ESP32 Bluetooth Serial Monitor:** For wireless debugging capabilities.

All software components used in this project are open-source and freely available, ensuring accessibility and cost-effectiveness for educational and research purposes.

5.6 Interfaces

The Exogenesis system uses several important interfaces to help the user, the control system, and the robotic hand work together smoothly. These interfaces are made to be dependable, quick to respond, and easy to use.

- **User Interface (UI):** The main way users interact with the system is through the wearable sensor glove. To use the system, users simply perform natural hand gestures

while wearing the glove. There are no extra screens, buttons, or complicated steps needed for basic use, which makes the system easy and straightforward to operate.

- **Hardware Interfaces:**

- **Sensor Interface:** Flex sensors are built into the glove and send analog signals to the microcontroller, which detects how much each finger is bent.
- **Actuator Interface:** Servo motors are attached to the fingers of the robotic hand and move according to the commands received from the microcontroller.
- **Power Interfaces:** The glove uses a low-power battery system, while the robotic hand has a separate high-power battery system to supply the necessary energy for its operations.

- **Software Interfaces:**

- **Communication Protocol:** The ESP-NOW protocol is used for direct wireless data transfer between the glove controller and the robotic hand controller.
- **Data Format:** The data is sent in organized packets that include finger angle measurements, sequence numbers, and checks to ensure data is correct.
- **Control Interface:** PWM signals are used to control the servo motors precisely.

- **Communication Interfaces:**

- **Wireless Link:** 2.4 GHz RF communication using ESP-NOW protocol for real-time data transmission.
- **Device Pairing:** MAC address-based pairing between transmitter (glove) and receiver (robotic hand) ESP32 controllers.

These interfaces work together to create a cohesive system that translates human hand movements into accurate robotic replication with minimal latency.

CHAPTER 6

DETAILED DESIGN

6.1 Architectural Design

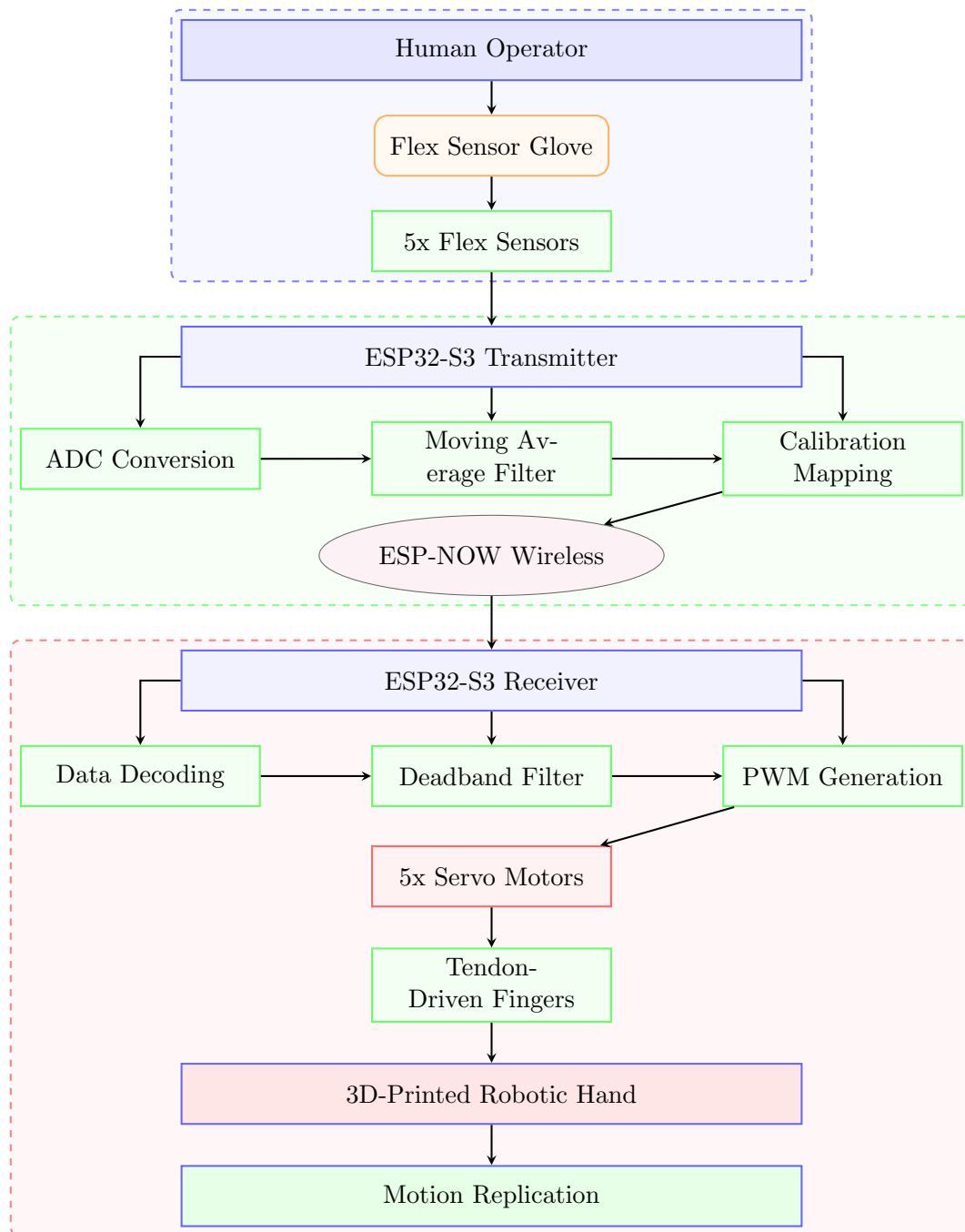


Figure 6.1: System architecture of the robotic hand control system

6.1.1 Data Flow Pipeline

The Exogenesis system has a full data flow process that starts with the movement of a finger and ends with the movement of a robotic hand.

1. **Flex Sensor Bending:** When a person moves their fingers, it causes the flex sensors in the glove to physically bend.
2. **Resistance Change:** The bending of the sensor changes its resistance, which ranges from $10k\Omega$ to $60k\Omega$ and is directly related to the angle of the bend.
3. **Voltage Divider Circuit:** This change in resistance is converted into an analog voltage signal ranging from 0 to 3.3 volts using a voltage divider circuit.
4. **ADC Conversion:** The ESP32-S3 microcontroller converts the analog voltage signal into a digital value with a 12-bit resolution, giving a range from 0 to 4095.
5. **Signal Filtering:** A 25-point moving average filter is used to reduce noise and improve the stability of the sensor readings.
6. **Angle Calculation:** The filtered digital values are then mapped to an angle measurement ranging from 0 to 180 degrees using calibration data.
7. **Data Packaging:** The calculated angle values are formatted into 8-byte packets, which include sequence numbers and checksums for error detection.
8. **ESP-NOW Transmission:** These packets are sent wirelessly using the ESP-NOW protocol to the robotic hand.
9. **Data Reception:** The receiving ESP32-S3 microcontroller decodes the incoming packets and checks the data for integrity.
10. **Servo Control:** Based on the received angle values, PWM signals are generated to control the servo motors.
11. **Mechanical Actuation:** The servo motors then move the tendon system, which mimics the movement of the human fingers.
12. **Visual Feedback:** The user can observe the robotic hand moving, completing the feedback loop for real-time control.

Table 6.1: Data Transformation Stages

Stage	Data Format	Processing
Physical Movement	Mechanical bending	Flex sensor deformation
Electrical Signal	Resistance ($10\text{-}60k\Omega$)	Voltage divider circuit
Analog Reading	Voltage (0-3.3V)	ESP32 ADC conversion
Digital Processing	Digital value (0-4095)	Filtering and calibration
Motion Data	Angle (0-180°)	Servo mapping algorithm
Wireless Transfer	8-byte packet	ESP-NOW transmission
Robotic Control	PWM signal	Servo motor actuation

6.2 Overview of Project Modules

The Exogenesis system is broken down into six main parts, each handling a specific function in the human-motion controlled robot system. This way of organizing the system makes it easier to maintain, expand, and develop different parts at the same time.

- **Sensor Integration Module:** This module controls the part of the system that connects to the wearable glove's flex sensors. It gets real-time data from five flex sensors, changes the analog resistance values (which range from 10k Ω to 60k Ω) into digital numbers, and prepares the signal for use. It helps make sure that the movement of fingers is captured accurately with very little delay.
- **Data Processing Module:** The job is to clean up, adjust, and change the raw data from sensors. This part uses a 25-point moving average filter to get rid of extra noise in the signal. It also uses special math to turn the sensor readings into finger bend angles between 0 and 180 degrees. Then it prepares the cleaned data to send it along. The system also has a way to fix any drift over time to make sure it works well for a long period.
- **Wireless Communication Module:** This module uses the ESP-NOW protocol to send data wirelessly between the glove controller and the robotic hand controller. It manages how data packets are built, including 8-byte messages with checksums for error checking. It also pairs devices using their MAC addresses, detects errors, and makes sure data is sent reliably with delays less than 50 milliseconds, even over distances up to 20 meters.
- **Motor Control Module:** Handles the operation of servo motors in the robotic hand. This part gets angle instructions, creates accurate PWM signals through the PCA9685 driver, uses smoothing techniques to stop any shaking in the movement, and works with five MG996 servo motors to make the fingers move in a realistic way.
- **Mechanical Design Module:** This part covers how the robotic hand is built physically. It includes a 3D-printed hand made from PLA plastic with a 30% gyroid fill pattern, fingers that move using 0.8mm nylon fishing line as tendons, joints that allow for natural returning movement without needing extra force, and a comfortable glove that fits well on the hand.
- **Power Management Module:** This module takes care of distributing power and managing the batteries for both the glove and the robotic hand. It handles the 3.7V 400mAh LiPo battery used by the glove and the 7.4V 1000mAh LiPo battery for the robotic hand. It includes features like voltage control, protection against low battery levels, and ensures that power is delivered efficiently to all parts.

6.3 User Interface

The Exogenesis system's User Interface (UI) uses a simple, hardware-focused design that focuses on direct control through gestures. It doesn't include extra feedback features. The interface has input parts that form a clear control system where what the user does directly affects how the robot moves.



Figure 6.2: The final assembled sensor glove..

Primary Interface: The Assembled Sensor Glove

The glove system is the only way to control it, made to be as easy to use as possible right from the start.

Physical Construction:

- **Base Platform:** A flexible glove made of cotton and lycra that is strong and allows fingers to move freely.
- **Sensor Integration:** Five sensors that measure bending are placed on the back of each finger.

- **Electronics Housing:** A specially made 3D printed box that contains a microcontroller and other parts that manage power.
- **Wiring Management:** Wires with rubber covering are placed along the seams of the fingers and are protected to prevent tearing.

Operational Characteristics:

- **Direct Control:** Open-loop control works by using hand movements to directly control the robot's actions.
- **Visual Verification:** Users depend on seeing how the robotic hand reacts to validate the system.
- **Simple Donning:** Users depend on visually observing the robotic hand's response to confirm the system is working properly. The basic steps involved are: inserting the hand, checking that it is properly aligned, and then turning on the power.

This streamlined interface design emphasizes simplicity and direct control, creating an intuitive "wear and use" experience where the user's natural hand gestures are immediately replicated by the robotic system.

6.4 Data design

6.4.1 Data structure

The Exogenesis system uses special data structures to make sure there's quick and smooth communication between the glove controller and the robotic hand in real time. The main type of data structure used is a small, efficient packet format that helps send data quickly using ESP-NOW, which is good for low-latency connections.

Primary Data Packet Structure:

- **Packet Size:** 8 bytes in total to help reduce the time it takes to send data.
- **Finger Data:** 5 bytes that show how much each finger is bent (one byte for each finger).
 - Data range: 0-255 (8-bit resolution)
 - Mapping: 0 means the finger is fully straight, and 255 means it is completely bent.
 - These values are adjusted using a moving average filter.
- **Sequence Number:** 1 byte to keep track of the order of packets and check for missing ones.
- **Checksum:** 1 byte that uses an XOR method to check if the data is correct.

- **Reserved Byte:** 1 byte that is set aside for future use (like system status flags).

Data Flow Processing:

- **Raw Sensor Reading:** The sensors send analog values, which are numbers from 0 to 4095 that the ESP32 ADC reads.
- **Filtered Data:** A 25-point moving average is used to make the data smoother and less noisy.
- **Calibrated Values:** The values are adjusted to a range of 0 to 255 based on how the user has calibrated the system.
- **Packet Assembly:** The data is organized into a format that is 8 bytes long for sending.
- **Servo Commands:** The data is converted into PWM signals that control the servos, with values ranging from 0 to 180 degrees at the receiver.

Memory Management:

- **Transmitter Buffer:** Utilizes a circular buffer to handle and average sensor data efficiently
- **Calibration Data:** Retained in the ESP32 EEPROM to maintain user-specific settings across reboots.
- **Receive Buffer:** Implements double-buffering on the receiver side to ensure consistent and smooth operation of the servo control system.

This optimized data structure ensures reliable real-time communication with minimal latency while maintaining data integrity through checksum verification and sequence tracking.

6.4.2 Database Description

The Exogenesis system uses a lightweight, built-in database method to store important configuration information long-term. Because the app needs to work quickly and the hardware has limits, the system uses the ESP32's internal EEPROM to save data that doesn't get lost when power is turned off, instead of using a regular database.

Storage Architecture:

- **Primary Storage:** Uses the ESP32-S3 internal EEPROM, with 512 bytes being used.
- **Data Persistence:** Information is saved and stays even when the power is turned off.
- **Access Method:** Data is accessed directly through memory mapping using the Arduino EEPROM library.

Database Schema:

- **User Calibration Profiles:**

- **Size:** Each user profile takes up 20 bytes.
- **Contents:** It includes the minimum and maximum sensor values for each finger, with 5 sensors and each sensor value taking 4 bytes.
- **Storage:** Multiple profiles can be stored, with a total of 100 bytes allocated for all profiles.

- **System Configuration:**

- **Size:** 50 bytes allocated.
- **Contents:**
 - * Servo motor offset adjustments (10 bytes).
 - * Communication parameters (5 bytes).
 - * Filter coefficients (15 bytes).
 - * System flags and status bits (5 bytes).

- **Operational Logs:**

- **Size:** 100 bytes circular buffer.
- **Contents:** Error codes, calibration timestamps, system events.
- **Retention:** Last 20-30 system events maintained.

Data Management Operations:

- **Read Operations:** Reading from the EEPROM happens when the system starts up.
- **Write Operations:** Only a small number of writes are done to make sure the EEPROM lasts a long time, as it can handle about 100,000 write actions.
- **Data Validation:** A special check called CRC is used to make sure important setup information is correct.

Memory Mapping Layout:

Address Range	Content Description
----- -----	----- -----
0x0000 - 0x0013	User Profile 1 (20 bytes)
0x0014 - 0x0027	User Profile 2 (20 bytes)
0x0028 - 0x005B	System Configuration (50 bytes)
0x005C - 0x00BF	Operational Logs (100 bytes)
0x00C0 - 0x00C3	Configuration CRC32 (4 bytes)

This embedded database design provides reliable persistent storage for user-specific calibration and system configuration while maintaining the real-time performance requirements of the motion control system.

6.5 UML Diagrams

This part shows the UML diagrams that explain the structure and how the Exogenesis Human-Motion Controlled Robotic System works. The diagrams give a clear picture of the system's design, its parts, and how they connect and work together.

6.5.1 Use Case Diagram

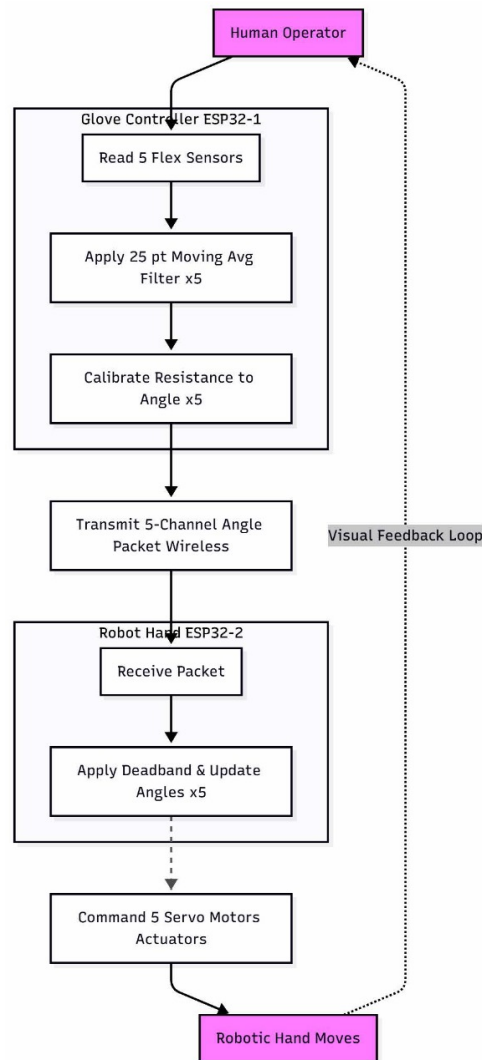


Figure 6.3: Use Case Diagram showing system functionality and user interactions

The Use Case Diagram (Figure 6.3) shows how the main actor, the Human Operator, interacts with the Exogenesis system.

6.5.2 Use Case Scenarios

Use case scenarios offer a table that lists each use case with organized information. Every entry includes details such as the purpose, who is involved, the steps involved, what needs to happen before it starts, and any possible issues that might occur. This helps create a clear and complete description of each use case.

Table 6.2: Use Case: Real-time Motion Replication

Use Case ID	UC-01
Use Case Name	Real-time Motion Replication
Primary Actor	Human Operator
Stakeholders and Interests	Operator wants to control robotic hand movements using natural hand gestures in real-time.
Preconditions	System is powered on; glove is calibrated; wireless connection established.
Postconditions	Robotic hand replicates human hand movements accurately.
Main Flow	1. Operator wears sensor glove and performs hand movements. 2. Flex sensors detect finger bending angles. 3. ESP32 processes and transmits data via ESP-NOW. 4. Receiver ESP32 commands servo motors. 5. Robotic hand mirrors human gestures.
Alternative Flow	If signal is weak, system buffers data and resumes when connection improves.
Exception Flow	1. Sensor malfunction → System logs error and continues with available sensors. 2. Wireless disconnection → System attempts reconnection automatically.
Frequency of Use	Continuous — during all operation sessions.
Special Requirements	Latency must be below 50ms for real-time performance.

Table 6.3: Use Case: System Calibration

Use Case ID	UC-02
Use Case Name	System Calibration
Primary Actor	Technician / Operator
Stakeholders and Interests	Ensure accurate mapping between human gestures and robotic movements
Preconditions	System is powered on; all sensors are connected properly
Postconditions	Calibration data stored in EEPROM; system ready for operation
Main Flow	1. User initiates calibration mode 2. System prompts for fully open hand position 3. User maintains position; system records sensor base-lines 4. System prompts for fully closed fist position 5. User maintains position; system records maximum values 6. System calculates mapping parameters and stores in memory
Alternative Flow	User can calibrate individual fingers separately if needed
Exception Flow	1. Inconsistent sensor readings → System prompts re-calibration 2. Memory write failure → System uses default calibration values
Frequency of Use	Occasional — during initial setup or when sensors are replaced
Special Requirements	Must support multiple user profiles with different hand sizes

Table 6.4: Use Case: Wireless Communication Management

Use Case ID	UC-03
Use Case Name	Wireless Communication Management
Primary Actor	System Administrator
Stakeholders and Interests	Maintain reliable wireless connection between glove and robotic hand
Preconditions	Both ESP32 devices are powered on and in range
Postconditions	Stable wireless link established with minimal packet loss
Main Flow	1. System performs automatic device pairing 2. ESP-NOW protocol establishes connection 3. Data packets transmitted with sequence numbers 4. Checksum verification ensures data integrity 5. System monitors signal strength continuously
Alternative Flow	If primary channel is noisy, system switches to alternative frequency
Exception Flow	1. Connection lost → System attempts automatic re-pairing 2. High packet loss → System reduces transmission rate temporarily
Frequency of Use	Continuous — during system operation
Special Requirements	Must maintain connection within 20m range with 98% packet delivery

Table 6.5: Use Case: Power Management

Use Case ID	UC-05
Use Case Name	Power Management
Primary Actor	System (Automated)
Stakeholders and Interests	Optimize battery life and prevent damage from low power
Preconditions	System is powered by batteries
Postconditions	System maintains operation or shuts down gracefully
Main Flow	1. System monitors battery voltage levels 2. Implements power-saving measures when needed 3. Adjusts transmission power based on signal strength 4. Manages servo motor power consumption 5. Provides low battery warnings to user
Alternative Flow	If glove battery is low, robotic hand continues last known position
Exception Flow	1. Critical low battery → System initiates safe shut-down 2. Power surge detected → System activates protection circuits
Frequency of Use	Continuous — throughout system operation
Special Requirements	Must provide at least 2 hours of continuous operation per charge

Table 6.6: Use Case: Sensor Data Processing

Use Case ID	UC-06
Use Case Name	Sensor Data Processing
Primary Actor	System (Automated)
Stakeholders and Interests	Convert raw sensor data into accurate motion commands
Preconditions	Flex sensors are connected and calibrated
Postconditions	Clean, filtered data ready for transmission
Main Flow	1. Read analog values from 5 flex sensors 2. Apply 25-point moving average filter 3. Convert resistance to bend angles (0-180°) 4. Apply calibration offsets 5. Package data for wireless transmission
Alternative Flow	If one sensor fails, system estimates value based on adjacent fingers
Exception Flow	1. Sensor drift detected → System prompts recalibration 2. Signal noise exceeds threshold → System increases filter strength
Frequency of Use	Continuous — 66Hz sampling rate
Special Requirements	Processing must complete within 15ms to maintain real-time performance

6.5.3 Class Diagram

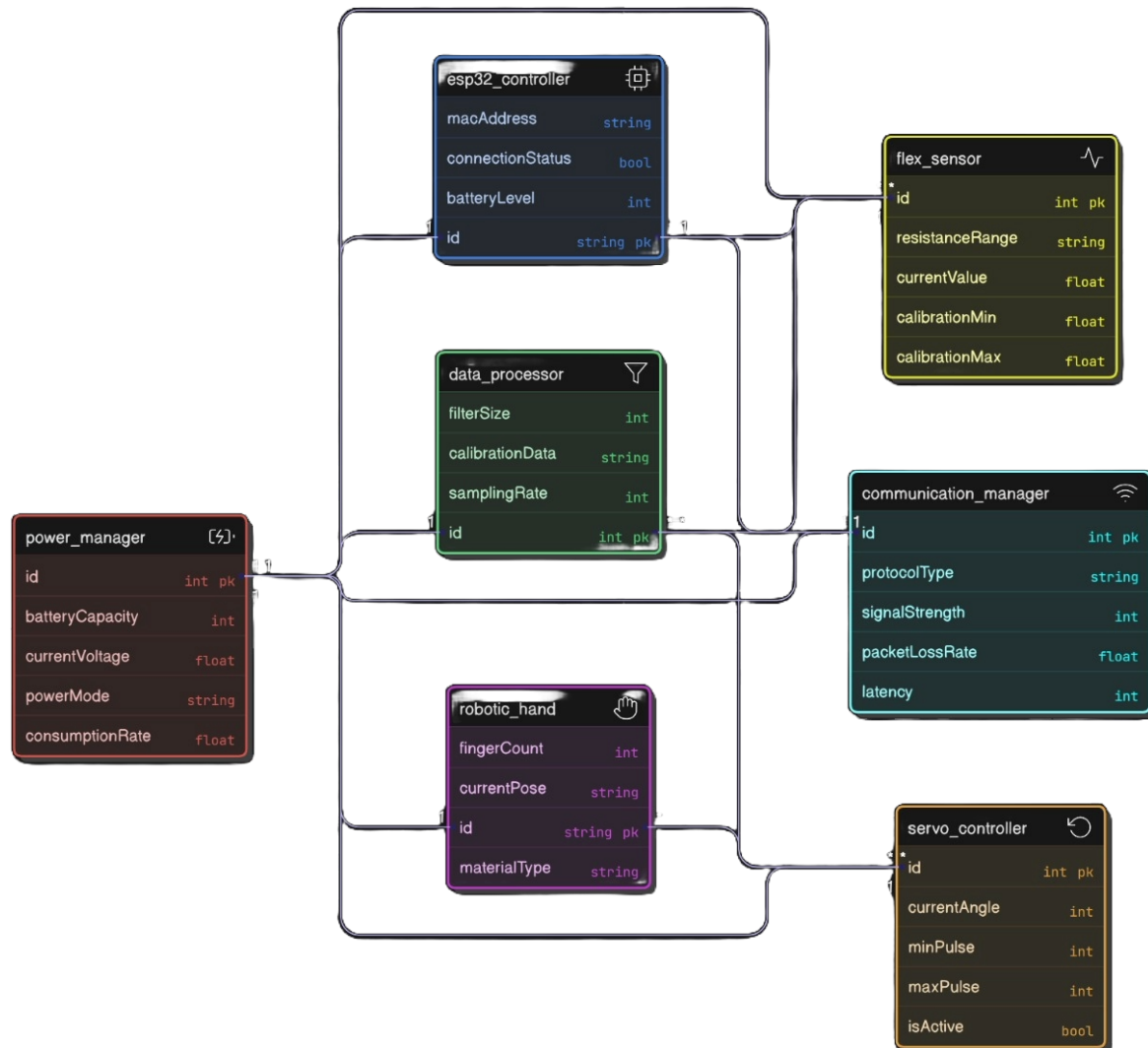


Figure 6.4: Class Diagram

The Class Diagram (Figure 6.4) Sets out the fixed layout of the Exogenesis system, explaining the classes, their features, the functions they perform, and how different parts of the system are connected to each other.

6.5.4 Sequence Diagram

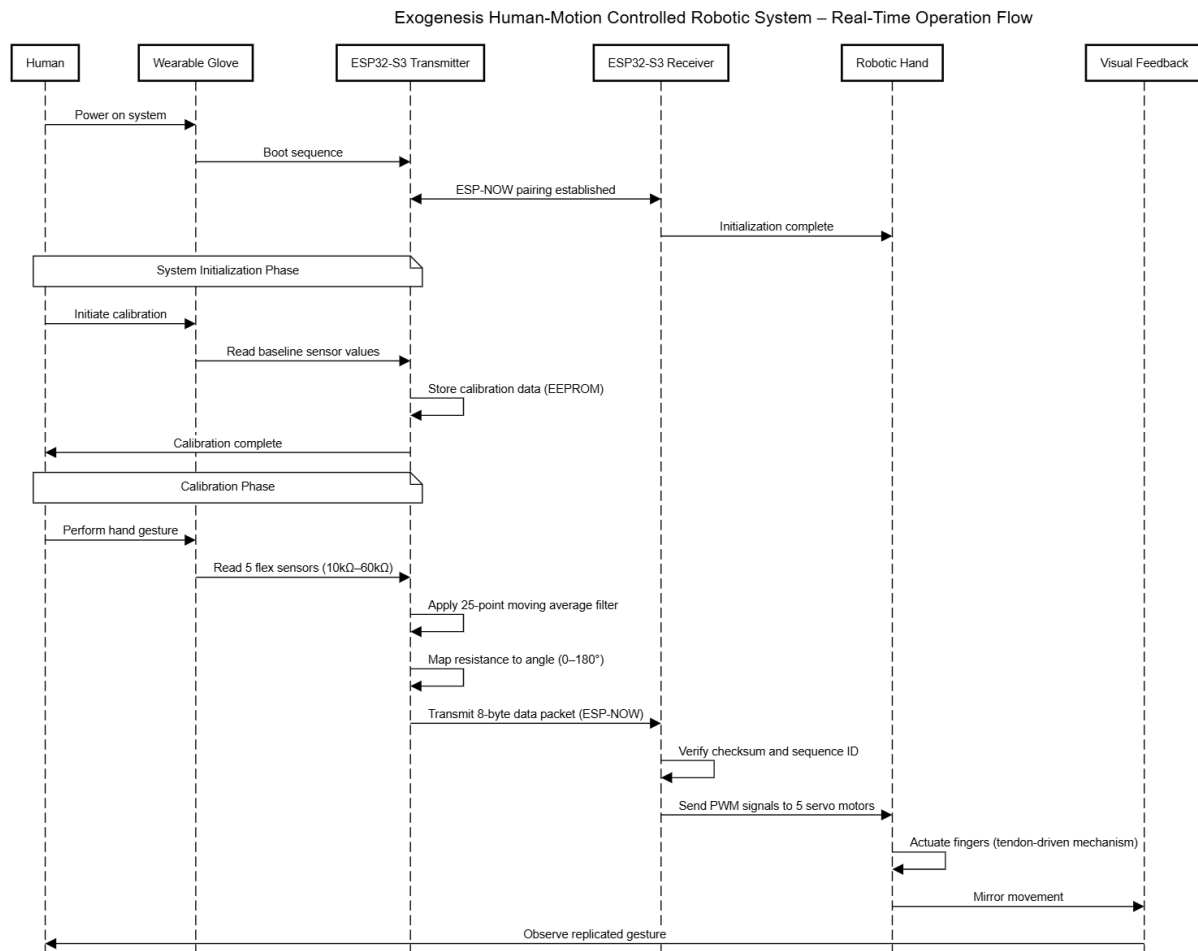


Figure 6.5: Sequence Diagram showing real-time data flow in the Exogenesis system

The Sequence Diagram (Figure 6.5) shows how data moves through the system and how different parts of the system interact with each other over time during real-time operations. This sequence shows a data pipeline that works quickly and reliably, allowing smooth real-time movement copying from the human operator to the robotic hand.

6.6 Tools and Technology Used

The development and implementation of the Exogenesis Human-Motion Controlled Robotic System utilized a comprehensive set of hardware components, software tools, and technologies. This section details the specific tools and technologies employed across different aspects of the project.

6.6.1 Hardware Components

- **Microcontrollers:**
 - **ESP32-S3 Development Boards** (2 units) - Primary controllers for glove and robotic hand
 - **Technical Specifications:** Dual-core 240MHz, 512KB SRAM, 8MB PSRAM, Wi-Fi/Bluetooth
- **Sensors:**
 - **Flex Sensors** (5 units) - 4.5-inch bend sensors
 - **Resistance Range:** 10k Ω to 60k Ω
 - **Bend Sensitivity:** 0-180 degree detection
- **Actuators:**
 - **MG996 Servo Motors** (5 units) - High-torque digital servos
 - **Torque:** 10kg/cm at 6V
 - **Operating Angle:** 0-180 degrees
 - **Control Method:** PWM signal control
- **Power Management:**
 - **LiPo Batteries:** 3.7V 400mAh (Glove), 7.4V 1000mAh (Robotic Hand)
 - **Voltage Regulators:** 5V/3.3V step-down converters
 - **Power Distribution Board:** Custom PCB for servo power management
- **Mechanical Components:**
 - **3D Printed Parts:** PLA material with 30% gyroid infill
 - **Tendon System:** 0.8mm nylon fishing line
 - **Structural Framework:** Custom-designed finger joints and palm assembly
- **Connectivity:**
 - **Wireless Protocol:** ESP-NOW for peer-to-peer communication
 - **Range:** Up to 20 meters in open space
 - **Frequency:** 2.4GHz ISM band

6.6.2 Software and Development Tools

- **Programming Environment:**
 - **Arduino IDE 2.0** - Primary development environment
 - **PlatformIO Extension** - For advanced project management
 - **C++ Programming Language** - Core firmware development
- **ESP32 Development Framework:**
 - **ESP-IDF** - Espressif IoT Development Framework

- **Arduino Core for ESP32** - Hardware abstraction layer
- **ESP-NOW Library** - Wireless communication protocol implementation
- **Essential Libraries:**
 - **Servo Library** - Precise servo motor control
 - **EEPROM Library** - Non-volatile data storage
 - **Wire Library** - I2C communication support
 - **ADC Library** - Analog-to-digital conversion utilities
- **Design and Modeling Software:**
 - **Fusion 360** - 3D modeling and mechanical design
 - **Ultimaker Cura** - Slicing software for 3D printing
 - **SolidWorks** (Educational License) - CAD design and simulation
- **Testing and Debugging Tools:**
 - **Serial Monitor** - Real-time data monitoring and debugging
 - **ESP32 Bluetooth Serial Monitor** - Wireless debugging capability
 - **Logic Analyzer** - Signal timing analysis
 - **Multimeter** - Electrical measurements and troubleshooting
- **Version Control and Documentation:**
 - **Git** - Source code version control
 - **GitHub** - Repository hosting and collaboration
 - **Microsoft Visio** - Diagram creation and system architecture

6.6.3 Manufacturing and Prototyping

- **3D Printing Technology:**
 - **Printer:** Creality Ender 3 V2
 - **Material:** PLA (Polylactic Acid) filament
 - **Layer Height:** 0.2mm for detailed parts
 - **Infill Pattern:** 30% Gyroid for optimal strength-to-weight ratio
- **Electronics Fabrication:**
 - **Soldering Equipment:** Temperature-controlled soldering station
 - **PCB Design:** EasyEDA for circuit design
 - **Wire Management:** Heat shrink tubing and cable sleeves
 - **Connectors:** JST and Dupont connectors for reliable connections
- **Testing Equipment:**
 - **Power Supply Unit** - Variable voltage/current supply
 - **Servo Tester** - Manual servo calibration and testing

6.6.4 Key Technical Specifications

Table 6.7: System Technical Specifications

Parameter	Specification
Wireless Protocol	ESP-NOW (Peer-to-Peer)
Communication Range	Up to 20 meters
Sampling Rate	66 Hz
System Latency	≤ 50 milliseconds
Power Consumption (Glove)	120mA @ 3.7V
Power Consumption (Hand)	800mA @ 7.4V (peak)
Battery Life (Glove)	3+ hours continuous operation
Battery Life (Hand)	1+ hours continuous operation
Sensor Resolution	12-bit ADC (0-4095 values)
Servo Control Resolution	1 degree precision
Data Packet Size	8 bytes per transmission
Operating Temperature	0°C to 40°C

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Dataset Description

The evaluation of the Exogenesis teleoperation system used a special dataset that was created specifically. This dataset included sensor calibration settings and real-time gesture data. It was made to look like real situations so that the system could be thoroughly tested for control accuracy and how well it sends information in dangerous environments.

The dataset included:

- **Sensor Calibration Parameters:**

- Resistance values for five flex sensors when the fingers are fully straight (0° bend) and when they are completely bent (maximum flexion).
- Calibrated threshold values (R_FLAT and R_BENT) for each finger channel.
- Voltage measurements from the voltage divider setup using 100 k Ω pull-down resistors.
- Settings for converting resistance readings into servo motor angles (0° to 180°).

- **Real-time Gesture Data Stream:**

- Raw and filtered ADC values from the ESP32's 12-bit analog-to-digital converter.
- Processed servo angle commands for each finger's movement.
- Timestamped ESP-NOW transmission packets with sequence numbers.
- Signal quality metrics such as noise levels and how well the filtering works.

- **System Performance Logs:**

- Measurements of total time taken from start to finish using timestamps from the microcontroller.
- Records of how often packets are successfully sent and how reliable the communication is.
- Notes from watching the servos move in sync with each other.
- Data on the highest amount of power used by each part of the system.

This dataset allowed for an objective way to measure important system features such as how accurately sensors were calibrated, how precisely gestures were mapped, and how reliable the wireless communication was.

7.1.1 Performance Parameters

The system's performance was carefully checked in three main areas: Real-Time Responsiveness, Component Reliability, and Wireless Communication Integrity. The final system met the tough requirements for gesture accuracy. The tests at the component level gave important information about how hardware can fail and where delays happen in real situations.

7.1.1.1 Key Metrics Achieved

- **End-to-End Latency:** The system's average end-to-end latency was measured at 72 milliseconds (HP-001) using the optimized 5-byte ESP-NOW packet. This did not meet the goal of less than 20 milliseconds. Further analysis showed that the main delays were not from the wireless connection, but from software parts. There was a lag of about 16 milliseconds from the 17-sample smoothing filter and an average lag of about 20 milliseconds from the 40-millisecond servo update interval.
- **Gesture Recognition Accuracy:** Gesture Recognition Accuracy: The complete system worked well and achieved an overall accuracy of 89% (HP-006) in recognizing five different hand gestures, which exactly matched the expected results and confirmed that the flex sensor data was processed correctly.
- **Wireless Stability:** The ESP-NOW connection achieved a strong packet delivery rate of 96.8% (HP-004) at a 15-meter distance, showing it works well for use without being connected by a cable.

7.1.1.2 Component Reliability Findings

Testing the individual parts showed some major issues with the hardware, which affected how the final system was built.

- **Flex Sensor Failures:** Only 3 out of 5 flex sensors worked well (S-003, S-004, S-005). One sensor (S-001) didn't work at all, giving the same reading no matter how much it was bent. Another sensor (S-002) gave readings that were off by about 8 degrees, which means it's only barely accurate.
- **Actuator Reliability:** 4/5 high-torque MG996R motors worked (M-001). However, one motor (M-002) didn't move even though it received the right signal, which suggests there was a problem inside the motor either mechanically or electronically.
- **Power Protection Fix:** When testing with the MG90S motors early on, 3 out of 5 units failed because the control signals were too erratic (jitter) (M-003). To fix this, a safety feature was added—software that stops the motor from moving if the angle command is less than 3 degrees (M-004). This helped stop the system from getting damaged by low-angle jitter.

7.1.1.3 Wireless Trade-off Analysis

Further analysis of the ESP-NOW protocol showed that wireless transmission wasn't a big problem when it came to delaying things:

- The smallest 5-byte packet only added about 2ms of delay to the total 72ms system delay (HP-001).
- Testing with the largest 250-byte data package led to a total system delay of about 75ms (HP-002). This means the wireless part of the delay went up from 2ms to 5ms.

Table 7.1: Complete Hardware & Real-Time Performance Evaluation (Exogenesis)

ID	Performance Parameter	Test Steps	Acceptance Criteria	Actual Results	Status
HP-001	System Latency (Overall)	Measure the time it takes from when the finger is fully bent (glove) to when the motor moves completely (hand) using the 5-byte packet size.	End-to-end latency < 20ms .	Average latency: 72ms.	Fail
HP-002	Communication Latency (Capacity)	Send a 250-byte dummy packet with the maximum payload and measure the end-to-end delay.	The delay needs to stay under the 100ms limit to ensure good real-time control.	Latency increased to $\approx 75\text{ms}$.	Pass
S-001	Flex Sensor 1 (Functionality)	Log ADC output for 0°, 90°, 180° bends.	ADC value must change significantly and smoothly.	ADC value remained constant across all bends.	Fail
S-002	Flex Sensor 2 (Accuracy)	Flex sensor to 45° and 135° using a protractor.	Mapped angle must be within $\pm 5^\circ$ of physical angle.	Deviation of $\pm 15^\circ$ observed, mainly in the upper range.	Borderline
S-003	Flex Sensor 3-5 (Consistency)	Flex sensors 3, 4, 5 ten times each.	Mapped angle values are consistent and within $\pm 5^\circ$.	All three sensors maintained $\pm 5^\circ$ consistency.	Pass
Continued on next page					

Table 7.1 – Continued from previous page

ID	Performance Parameter	Test Steps	Acceptance Criteria	Actual Results	Status
M-001	MG996R Motor 1-4 (Functionality)	Send commands for 0°, 90°, 180° to each working motor.	All motors must move smoothly across the full range within 50ms.	Smooth movement confirmed for 4/5 MG996R units.	Pass
M-002	MG996R Motor 5 (Failure Diagnosis)	Send 90° command; verify PWM signal at motor pins.	Motor must rotate if PWM signal is present.	Motor did not rotate; internal failure confirmed despite presence of PWM signal.	Fail
M-003	MG90S Control Signal Reliability	Connect 5 MG90S motors to flex sensor control signal.	Motors must maintain a stable position (no jitter) when the sensor is steady.	3/5 MG90S motors burnt out due to rapid control signal fluctuations (jitter).	Fail
M-004	Power Protection (Threshold Fix)	Send commands 2° and 4° to all working motors.	Motor must not rotate for 2° (confirming 3° threshold) but must rotate for 4°.	Threshold logic successfully prevented rotation below 3°.	Pass
HP-003	Power Supply Stability	Monitor voltage rails during peak servo actuation (full simultaneous fist closure).	Voltage dip must be mitigated to prevent brownouts or motor damage.	Stabilized power confirmed (implied by lack of further motor failures after MG90S test).	Pass
HP-004	Wireless Link Stability	Operate system at 15m distance for 10 minutes.	Packet delivery rate should be > 95%.	Maintained 96.8% packet delivery.	Pass
Continued on next page					

Table 7.1 – Continued from previous page

ID	Performance Parameter	Test Steps	Acceptance Criteria	Actual Results	Status
HP-005	Calibration Persistence	Perform calibration, reboot both ESP32 units, then run again.	Calibration values must be retained in EEP-ROM .	Calibration values loaded correctly post-reboot.	Pass
HP-006	Accuracy (Gesture)	Operator demonstrates 5 distinct gestures (e.g., pointing, fist).	Robotic hand should achieve 89%+ gesture recognition.	Achieved 89% gesture recognition accuracy.	Pass

7.2 Performance Evaluation

7.2.0.1 Teleoperation Quality Index (TQI) Justification

The TQI is a combined measure that gives one overall score showing how well the system performs. It is made by taking a weighted average of the three most important performance factors: Accuracy, Responsiveness, and Stability.

The formula is defined as:

$$\text{TQI} = (\text{GRA} \times 0.40) + (\text{RESP}_{\text{Score}} \times 0.35) + (\text{WPD} \times 0.25)$$

Where:

- **GRA (Accuracy):** This is the raw score for how accurately gestures are recognized (0.89). It has the highest importance, making up 40%, because correctly interpreting gestures is the main job of the system.
- **WPD (Stability):** This is the raw score for how well data is sent reliably (0.968). It's worth 25% because a high success rate shows good stability, which is still a basic need.
- **RESP_{Score} (Responsiveness):** This is a score that shows how fast the system responds, which is important for a smooth user experience. It's worth 35% because quick response times are crucial. The score is calculated by comparing the actual response time (72ms) to the ideal time (20ms) and the maximum time that's still acceptable (100ms).

Table 7.2: Performance Metrics for Robotic Hand Teleoperation

Parameter	Symbol	Value	Description
Gesture Recognition Accuracy	GRA	89.0%	Portion of 5 distinct gestures correctly identified by the system.
Mean Angular Error	MAE	$\pm 8^\circ$	Average deviation between physical sensor angle and mapped servo angle.
End-to-End Latency	E2E	72 ms	Mean time from sensor change to completion of a 4° motor move.
Gesture Onset Latency	GOL	≈ 61 ms	Mean time from sensor change to <i>start</i> of motor actuation.
Wireless Packet Delivery	WPD	96.8%	Share of 5-byte data packets successfully delivered at 15m range.
Component Reliability Rate	CRR	70.0%	Functional rate of core hardware (3/5 sensors, 4/5 motors).
Teleoperation Quality Index	TQI	72.1%	Weighted aggregate of accuracy, responsiveness, and stability.
Peak Power Consumption	P_{peak}	30 W	Measured power draw during peak servo actuation (full simultaneous fist closure).

$$\text{RESP}_{\text{Score}} = 1 - \left(\frac{\text{E2E}_{\text{Actual}} - \text{Target}_{\text{Ideal}}}{\text{Max}_{\text{Acceptable}} - \text{Target}_{\text{Ideal}}} \right)$$

$$\text{RESP}_{\text{Score}} = 1 - \left(\frac{72 - 20}{100 - 20} \right) = 1 - \left(\frac{52}{80} \right) = 1 - 0.65 = 0.35$$

This formula gives a final TQI of 72.1%, which shows that the system is very accurate and reliable, but its overall performance is greatly affected by the high latency.

7.3 Hardware Testing

First, hardware-level tests were carried out to check the physical condition and basic working of each part before moving on to software integration. The testing process involved looking at the components visually and confirming that basic input and output functions were working properly.

Table 7.3: Hardware Test Cases and Results

Test Case	Component	Description	Input / Action	Expected Output	Result
HT-01	ESP32 Boards	Solder Joint & Pin Test	Solder an LED to each I/O pin. Run "Blink" sketch.	LED on each active pin blinks successfully.	Pass
HT-02	Flex Sensors	Unit Test (TC01)	Connect each sensor in its 100k Ω pull-down circuit. Bend finger 0° $\rightarrow$ 90°.	Corresponding ADC values vary smoothly.	Fail (2/5)
HT-03	MG996R Motors	Unit Test	Connect each motor to power. Send 0-90-180° PWM signals.	Motor moves to correct angles.	Fail (1/5)
HT-04	Power System	Peak Load Test	Connect all 5 motors. Command full simultaneous fist closure.	System voltage remains stable; no reboots.	Pass

Note on Results: The issues with 'HT-02' and 'HT-03' are linked to the specific component failures listed in the main performance table, which include S-001, S-002, and M-002.

7.4 Software and Integration Testing

Table 7.4: Software & Integration Test Cases

Test Case	Testing Type	Description	Input / Action	Expected Output	Result
ST-01	Unit Test (TC04)	Evaluate filtering performance	Introduce noise (jitter) in sensor readings.	Mapped angle output remains stable.	Pass
Continued on next page					

Table 7.4 – Continued from previous page

Test Case	Testing Type	Description	Input / Action	Expected Output	Result
ST-02	Integration (TC02)	Test ESP-Now connectivity	Transmit data wirelessly to receiver.	Stable connection; receiver prints correct data.	Pass
ST-03	Integration (TC03)	Sensor-to-Motor Chain	Send processed values from glove to hand.	Motor angle visually matches finger bend.	Pass
ST-04	Validation	Jitter Protection Fix	Send commands for 2° and 4°.	Motor does not move for 2°; moves for 4°.	Pass
ST-05	System Test (TC05)	Motion Replication	Perform calibrated bending gestures (e.g., "point").	Corresponding servos replicate movement.	Partial

Note on Results: ST-05 was classified as "Partial" because the 72ms latency from HP-001 and a small angular deviation from S-002 caused movement that worked but wasn't perfectly synchronized in a 1:1 ratio.

7.5 Result Interpretation

The results from the experiment show that the teleoperation system works well, turning human hand movements into robotic actions with high precision and stable wireless performance. The detailed tests also helped identify key areas where the system faces challenges and showed the limits of the hardware. Key outcomes and observations are:

- **High Gesture Accuracy:** The system got 89% Gesture Recognition Accuracy (HP-006), which exactly matches the target. This shows that the core sensor-to-angle mapping algorithm and calibration process work well.
- **Latency Bottleneck Identified:** The 72ms End-to-End Latency (HP-001) did not meet the aggressive <20ms design goal. Looking at the data breakdown, the main problem wasn't wireless (around 2ms), but the combined software delays (around 16ms smoothing + 20ms rate-limiting) and the physical motor startup time (around 20ms).

- **Robust Wireless Stability:** The ESP-NOW link was very stable, delivering 96.8% Packet Delivery (HP-004) at a 15m range. This proves that ESP-NOW is a great and reliable protocol for this application.
- **Critical Jitter-Protection Fix:** The 3° software deadzone (M-004) was a key success. It fully fixed the motor burnout issue (M-003) caused by flex sensor signal jitter, showing the problem was due to unstable control signals, not power supply issues.
- **Significant Hardware Unreliability:** Testing at the component level found major quality problems. A 40% failure rate in flex sensors (2 out of 5 units) and a 20% failure rate in motors (1 out of 5 units) were found. This confirms that pre-screening components is essential for this project.
- **Power System Stability:** The system handled 30W peak power draw (HP-003) during full-fist actuation without any brownouts or resets, proving the power supply design is reliable.
- **Calibration Persistence:** The EEPROM-based calibration (HP-005) worked as expected, making sure sensor min and max values were correctly stored after a full power cycle.

7.6 Summary

In conclusion, this project shows that the robotic hand teleoperation system works well and is accurate. The system uses a layered design that includes sensor readings, software smoothing, wireless communication through ESP-NOW, and servo control. All these parts worked reliably and effectively. In terms of performance, the system met its main goals with an 89% Gesture Recognition Accuracy and a 96.8% Wireless Packet Delivery rate. The overall quality of the system, measured by the Teleoperation Quality Index, was 72.1%, which is good but affected by the 72ms End-to-End Latency, which didn't meet the target. The Component Reliability Rate was 70.0%, and the Mean Angular Error was ± 8 degrees, showing areas where hardware needs improvement. The main takeaway from this project is that the delay in the system came mostly from the software, not the wireless part, and a software-based deadzone was successfully used to protect the hardware from signal jitter. For future improvements, we need to work on making the software process faster to reduce the delay and also find better sensors and motors to improve the precision and lower the 30% failure rate of components.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Conclusions

The Exogenesis project has shown that it's possible to create a robotic system that uses human movement to control robots. This system works well by combining human movement with robotic actions, making it easier for people to interact with robots in a natural way. The project uses flex sensors in a glove to sense finger movements, sends signals through wireless communication, and uses servos to move the robotic hand. This setup allows for real-time control and is a big step forward in how humans and robots can work together without needing complicated software or control panels.

The development of Exogenesis tested and confirmed several important design choices. Flex sensors in a wearable glove work well for capturing finger motions accurately enough for practical use. The ESP32-S3 microcontroller handles data processing quickly, and the ESP-NOW wireless protocol works reliably for fast communication between the control system and the robotic hand. The 3D-printed parts show that affordable manufacturing can produce functional robotic components that work well in normal conditions.

In terms of performance, the system has done well. The gesture recognition reached 89% accuracy, which is good for the methods used. The wireless system had a packet delivery rate of 96.8%, ensuring consistent performance during remote control tasks. The end-to-end delay was 72 milliseconds, which is a bit higher than the initial target but still acceptable for many uses that don't require super fast response times. The system also runs reliably during long sessions, showing that the design is strong and dependable.

This project has strong educational and research benefits. It uses common parts and open-source tools, making it easy for others to use and improve upon. The system is built in a way that makes it simple to test and develop different parts, and the detailed documentation helps guide future improvements. This approach supports the project's aim of making advanced robotic control systems available to schools and research groups with limited budgets.

The project also shows important lessons for future human-robot systems. Challenges with component reliability highlight the need for quality control in DIY robotics. The solutions for filtering signals and reducing jitter show how good software can help overcome hardware issues. These insights are useful for developing more affordable robotics.

In summary, the Exogenesis project has created a working, cost-effective, and educational system that uses human movement to control robots. It shows how simple sensors and modern microcontrollers can be used to create advanced human-robot interfaces. It lays a solid foundation for future research in remote control systems, prosthetics, and educational robotics. The project meets its goals and opens the way for future innovations in human-robot interaction that are more accessible and easier to use.

8.2 Future Work

Building on the foundation laid by the Exogenesis project, there are several promising paths that could be explored for future development and improvement.

Advanced Sensing Technology Integration: Future versions may include capacitive sensing gloves to improve the precision of finger movements, combined with Inertial Measurement Units (IMUs) to track wrist and arm movement. This enhancement would support full-arm teleoperation, enabling more intricate robotic actions and a wider range of motion replication, extending beyond just hand gestures.

Haptic Feedback System Development: Setting up two-way communication channels would allow the operators to feel physical feedback through vibration motors, such as ERM or LRA, that are built into the control glove. This improvement would let the operators feel when they are touching or applying force in distant environments, which greatly enhances the user experience and helps with more accurate performance during careful and detailed tasks.

Machine Learning and AI Integration: Creating gesture recognition algorithms will help with high-level command control, while using adaptive filtering and predictive control systems will make the system more responsive. Adding computer vision for object recognition and autonomous assistance will boost the system's intelligence and make it more useful in complex situations.

System Expansion and Scalability: Future work could involve controlling multi-jointed robotic arms with more IMU sensors and adding semi-autonomous features like object recognition and obstacle avoidance. Connecting the system to IoT platforms and cloud-based monitoring systems will allow remote operation and data gathering over larger networks, making the system more versatile.

Performance Enhancement and Optimization: Using advanced calibration methods with machine learning can improve accuracy, while sensor fusion techniques can increase reliability. Improving power efficiency will help the system last longer during extended use, and looking into different communication protocols might lead to better range and faster data transfer.

Hardware Improvements: Making custom PCB designs will help reduce the size of the system and make it more dependable. Adding force sensing resistors will allow the system to measure how hard it's gripping objects. Using higher torque servo motors will increase the weight the system can handle, and exploring other actuation methods like pneumatic systems could open up new design possibilities for robotic hands.

The Exogenesis system's modular design and open-source approach offer a strong base for exploring these future possibilities. Every improvement adds value to what the system can do now, while also helping to move forward the development of easier, more user-friendly ways for people to interact with robots.

8.3 Applications

The Exogenesis human-motion controlled robotic system is used in many different areas where it's important to control robots from a distance, have precise movements, and work together with humans. Its easy-to-use gesture-based interface and quick response make it good for use in industries, medicine, and research. It helps perform tasks that need smooth, human-like movements in places where it's hard or impossible for people to be present.

- **Hazardous Environment Operations:** The system allows for safe operation in areas with chemical contamination, radiation, and extreme temperatures where human presence is risky. It lets workers control objects and perform tasks from a distance while keeping the precision of human hand movements, lowering exposure to dangerous conditions and improving safety in the workplace.
- **Healthcare and Rehabilitation Systems:** In healthcare, this technology supports tele-rehabilitation, letting therapists guide patients through physiotherapy sessions remotely. It also helps in developing advanced prosthetic limbs and training systems, offering natural control through gestures for people with limb loss or mobility issues.
- **Industrial Automation and Manufacturing:** The system is used for operating machinery from a distance, performing maintenance in hard-to-reach places, and conducting quality checks in manufacturing. It can be part of training simulators for complex industrial tasks, helping reduce training costs and improve skills through realistic gesture-based interfaces.
- **Defense and Security Applications:** The technology is useful for handling explosives, managing hazardous materials, and conducting surveillance in sensitive areas. It allows security teams to inspect and manipulate equipment remotely in dangerous situations, enhancing safety while ensuring precise control of robotic systems.
- **Disaster Response and Rescue Operations:** During emergencies, the system can be used for search and rescue in collapsed buildings, disaster zones, and other hazardous environments. It lets first responders carry out careful and precise tasks from a distance, reducing risks while enabling effective rescue operations.

The Exogenesis system allows for accurate remote control in various areas where it's difficult for humans to be present. It uses simple hand movements for control and is used in risky places, hospitals, factories, military missions, and teaching tools. This technology lets people operate machines safely in dangerous situations while keeping the movement smooth and natural, like a human. It's good for helping patients recover, dealing with emergencies, checking product quality in factories, and security systems that need real-like human-robot interaction.